

OpenClaw Color Line Patrol

OpenClaw Color Line Patrol

1. Course Content
 - Learning Objectives
2. Preparation
3. Demonstration Case
4. Source Code Analysis
 - FollowLine
5. Parameter Tuning
 - 5.1 Color Calibration

1. Course Content

Learning Objectives

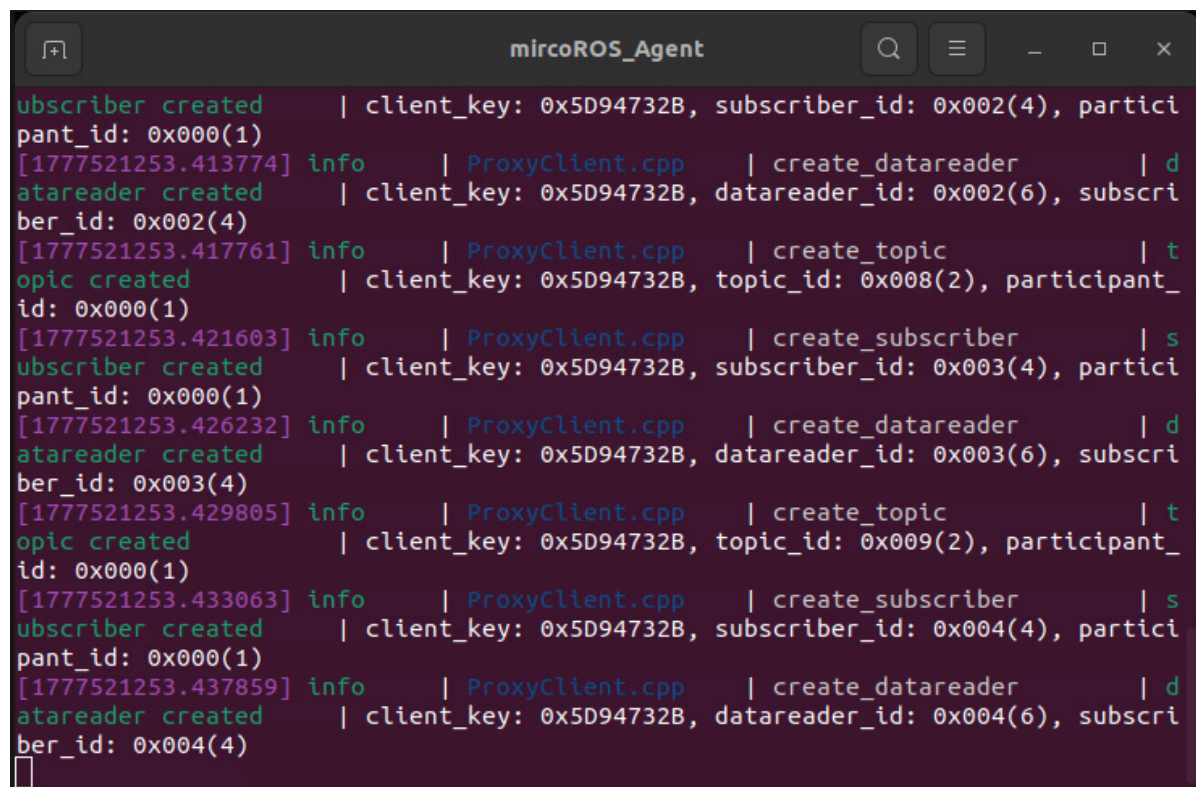
Through this chapter, you will master the following skills:

- **Master the `FollowLine` Tool Invocation:** Learn to use the MCP tool to start an automatic line-following task for a specified color (supports red, green, blue, yellow four colors).
- **Gain Color Calibration Ability:** Be able to use the `colorSelect` tool to adjust HSV parameters based on actual ambient lighting, optimizing recognition robustness.

2. Preparation

- Start the MicroROS chassis agent (no need to restart if already started)

```
sh start_agent.sh
```



```
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1777521253.413774] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x5D94732B, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
[1777521253.417761] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x5D94732B, topic_id: 0x008(2), participant_
id: 0x000(1)
[1777521253.421603] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x003(4), partici
pant_id: 0x000(1)
[1777521253.426232] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x5D94732B, datareader_id: 0x003(6), subscri
ber_id: 0x003(4)
[1777521253.429805] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x5D94732B, topic_id: 0x009(2), participant_
id: 0x000(1)
[1777521253.433063] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x004(4), partici
pant_id: 0x000(1)
[1777521253.437859] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x5D94732B, datareader_id: 0x004(6), subscri
ber_id: 0x004(4)
█
```

- Start odometry, tf, robotic arm assistance, camera nodes, etc.

```
ros2 launch m3pro_bringup car_base.launch.py
```

- Start the MCP service

```
ros2 launch m3pro_bringup mcp_service.launch.py
```

- If you need voice reply, you need to start openclaw_bridge additionally, otherwise it can be omitted

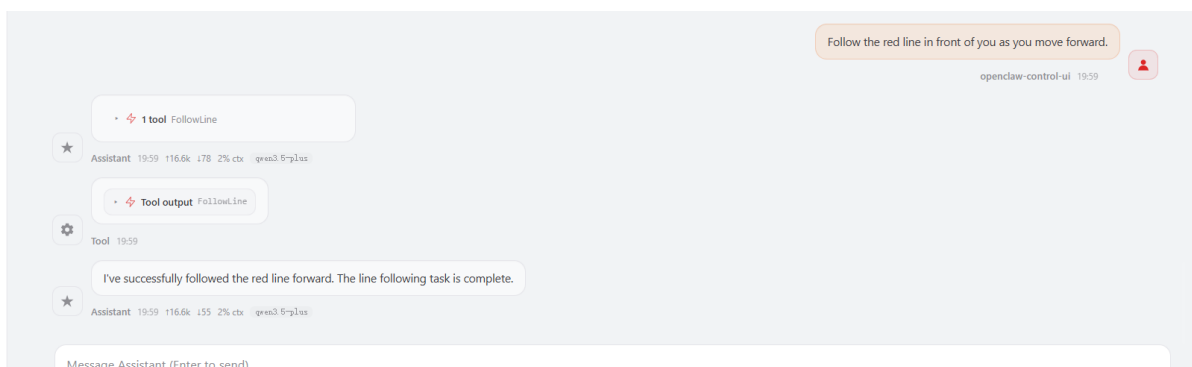
```
ros2 run multi_brains_pre openclaw_bridge
```

3. Demonstration Case

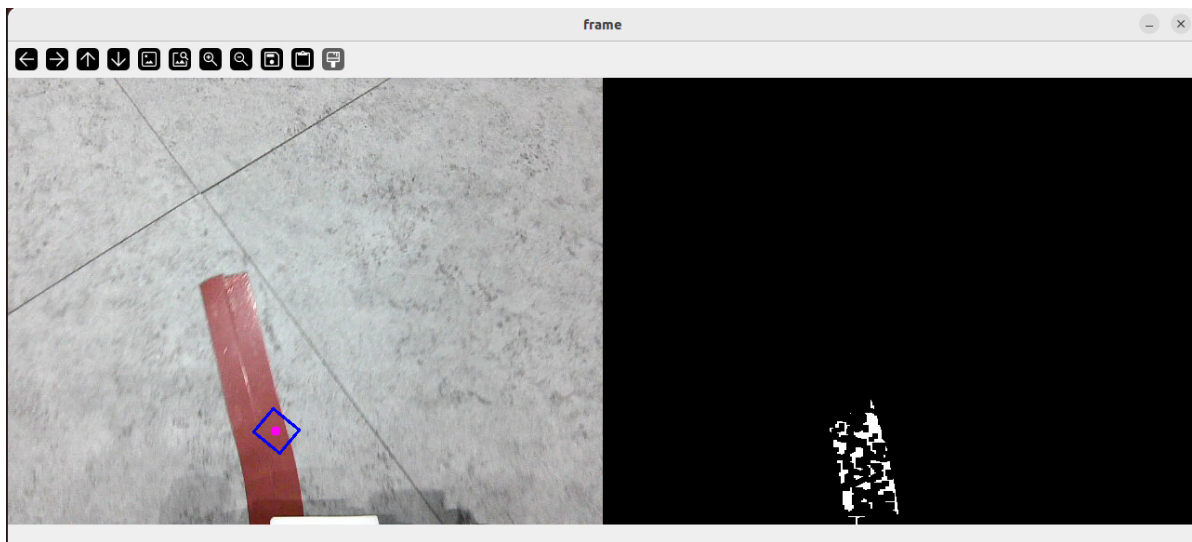
[!TIP]

- You can write commands according to your own needs. The following is a case demonstration. Choose any interaction method. Here we take the web-based WebChat as an example.

Command	Parameters	Function
FollowLine	<code>--color</code> line color, optional: red, green, blue, yellow	Drive along the ground color line route



- When the `FollowLine` tool is called, the system will pop up a display window showing the RGB raw image and the binarized processed image in real time, making it easy to observe recognition results.



4. Source Code Analysis

- Source path

```
$HOME/M3Pro_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

FollowLine

This function is responsible for starting and monitoring the underlying color line-following node.

```
def FollowLine(self,color:str) -> bool:
    import subprocess
    # 1. Adjust the robotic arm to the detection pose, ensuring the camera
    faces the ground ahead
    self._pubSix_Arm(self.waste_detect_pose)

    self.follow_line_future = Future()
    color = color.strip("\ ")

    # 2. Map the natural language color description to the numeric code
    recognized by the underlying node
    if color == 'red':
        target_color = float(1)
    elif color == 'green':
        target_color = float(2)
    elif color == 'blue':
        target_color = float(3)
    elif color == 'yellow':
        target_color = float(4)
    else:
        error_msg = f"Invalid color specified: '{color}'. Valid options are:
        red, green, blue, yellow."
        self.get_logger().error(Fore.RED+error_msg+Fore.RESET)
        return [False, error_msg]

    # 3. Start the line-following node as a subprocess, passing the color
    parameter
    self.follow_line_process = subprocess.Popen(['ros2', 'run',
    'multi_brains_pre', 'follow_line', '--ros-args', '-p', f'color:={target_color}'])
```

```

        # 4. Block and wait for the line-following task to complete (status
        feedback from underlying node via Future mechanism)
        while not self.follow_line_future.done():
            time.sleep(0.1)
        self.follow_line_clear_future = None

    # 5. After the task ends, clean up the subprocess and reset the robotic
    arm
    kill_process_tree(self.follow_line_process.pid)
    self.InitArmPose()
    return [True, "Follow line finished"]

```

Core Logic Description:

1. **Pose Preparation:** First, call `_pubSix_Arm` to move the robotic arm to the preset observation position, ensuring the camera's field of view covers the road ahead.
2. **Parameter Mapping:** Convert the user-input string color (e.g., "red") to the numeric ID agreed upon by the underlying algorithm for HSV threshold matching.
3. **Process Management:** Use `subprocess.Popen` to independently start the `follow_line` node. This design allows the line-following algorithm to run at high frequency in a separate process without affecting the main controller's response speed.
4. **Synchronization and Cleanup:** Use the `Future` object to synchronize the main program with the subprocess. When the line-following task ends (e.g., detecting the endpoint or manual termination), automatically kill the subprocess and restore the robotic arm to its initial state.

5. Parameter Tuning

- Parameter file path:

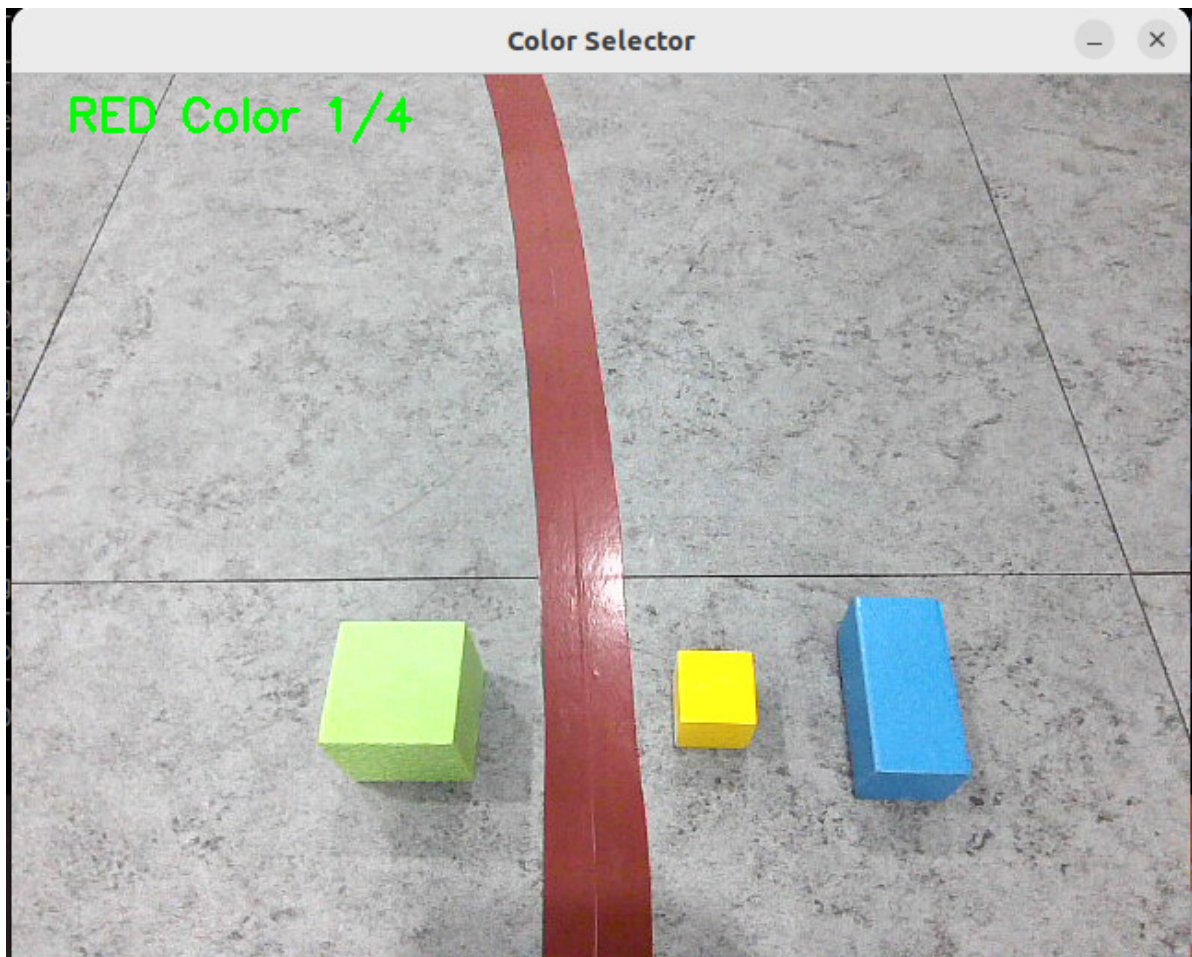
```
$HOME/M3Pro_ws/src/m3pro_bringup/config/followlineHSV.text
```

5.1 Color Calibration

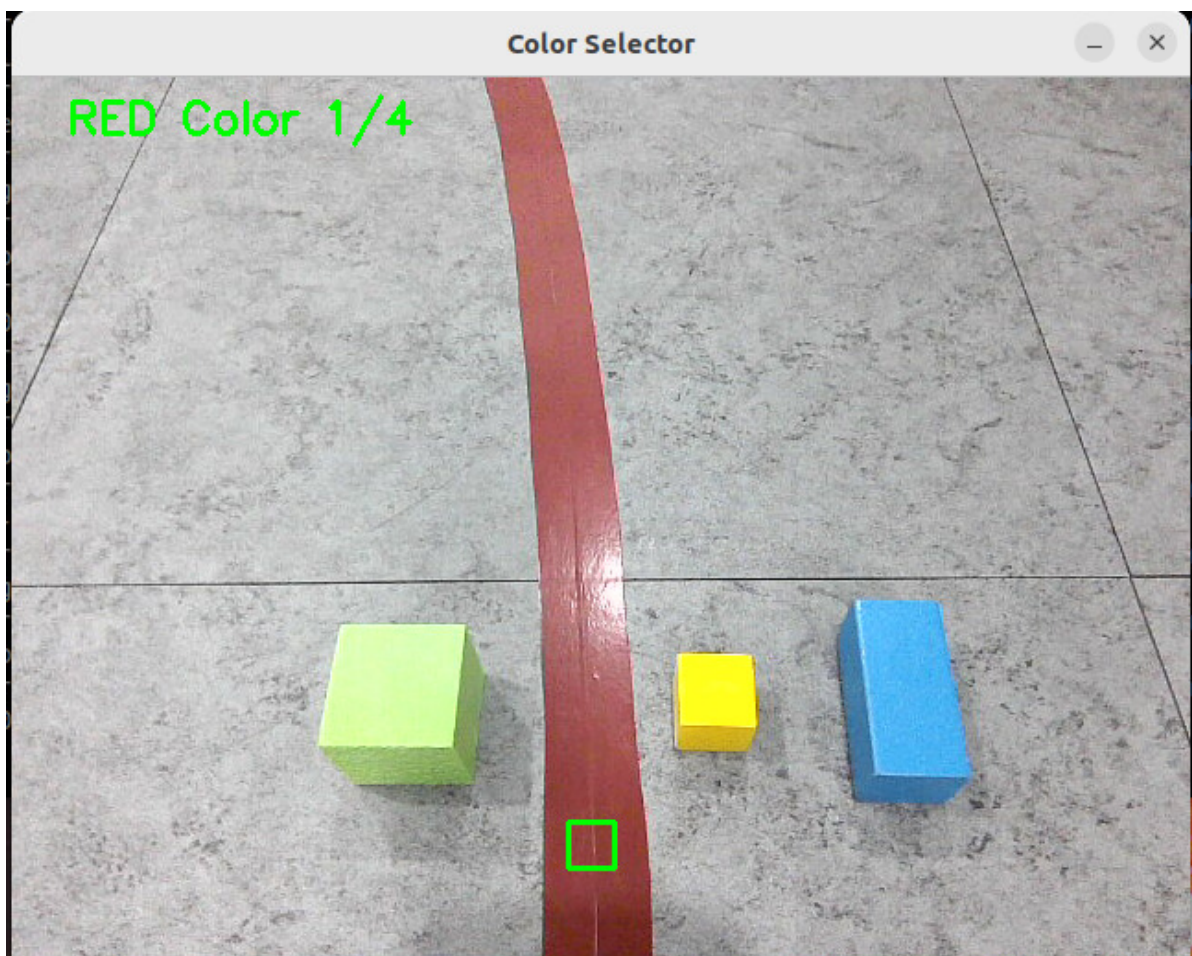
If the color recognition effect is poor during line following, resulting in inability to properly track the path, the HSV threshold for color recognition needs to be recalibrated. Basic calibration has been performed at the factory; if ambient lighting has not changed significantly, no additional operation is usually required.

```
ros2 launch m3pro_bringup colorSelect.launch.py
```

- After the program starts, a **Color Selector** window will appear.



- Use the mouse to select a region (the region should contain only one color)



- After confirming the color calibration is correct, press the **Spacebar**, and the red HSV value will be written to the parameter file. After calibration is complete, press Ctrl+C to exit the

program.